
Outils ETL (& ELT) - Projet

Ynov - M2 DATAE+IA

Geoffroy Ladrat

2025 - 2026



Table des matières

1	Le dilemme du prisonnier itératif : générer, transformer et comprendre la coopération	2
1.1	Préambule	2
1.2	Contexte	2
1.3	Sujet	2
1.4	Consignes	2
1.4.1	Génération des données (Extract / Generate)	2
1.4.2	Chargement et restitution (Load)	3
1.4.3	Transformation des données (Transform)	3
1.4.4	Agrégation et exposition	3
1.4.5	Choix d'architecture	3
1.4.6	Aller plus loin	4
1.4.7	Objectif final	4
1.5	Livrables	4
1.6	Conclusion	4

1 Le dilemme du prisonnier itératif : générer, transformer et comprendre la coopération

1.1 Préambule

Pour comprendre le contexte complet de ce sujet, il est conseillé de regarder cette vidéo (9min) :

- <https://www.youtube.com/watch?v=G9ER5bLxQEU>

1.2 Contexte

En 1981, le politologue Robert Axelrod organisa une expérience restée célèbre : un tournoi informatique où s'affrontaient des programmes incarnant différentes stratégies de comportement dans une situation appelée *le dilemme du prisonnier itératif*.

Cette situation modélise une tension universelle : faut-il coopérer avec l'autre au risque d'être trahi, ou trahir pour éviter d'être exploité ?

L'expérience d'Axelrod révéla un résultat inattendu : la stratégie la plus simple et la plus coopérative, *Tit for Tat* (donnant-donnant), surclassa toutes les autres. Elle montrait qu'à long terme, la coopération pouvait émerger spontanément dans un environnement d'agents égoïstes.

Quarante ans plus tard, vous allez rejouer cette expérience - mais à l'ère de la donnée et de l'intelligence artificielle.

1.3 Sujet

Vous devez concevoir un **pipeline de data ingénierie complet** autour d'une **simulation du dilemme du prisonnier itératif**, en y intégrant une **dimension générative** :

- En simulant les comportements à l'aide de stratégies codées,
- En déléguant les décisions à des agents d'IA locaux (par exemple via Ollama).

L'objectif est de produire un **jeu de données original**, propre, structuré et analysable, retraçant l'évolution des comportements de coopération ou de trahison entre agents.

1.4 Consignes

1.4.1 Génération des données (Extract / Generate)

- Simulez un tournoi entre plusieurs stratégies & agents IA.

- Chaque partie comporte un nombre défini de tours (par exemple 2000).
- Chaque tour enregistre : les choix des deux joueurs, leurs gains, et toute information pertinente (profil, score cumulé, justification éventuelle de l'IA).
- Vous devez coder vos stratégies (ex : *always cooperate*, *always defect*, *tit for tat*, *random*, *grim trigger*, etc.) ou les définir via des prompts IA (profil empathique, calculateur, rancunier...).

1.4.2 Chargement et restitution (Load)

- Chargez vos données dans un dataset local (Parquet).

1.4.3 Transformation des données (Transform)

- Nettoyez et structurez vos données.
- Calculez des indicateurs : taux de coopération, score moyen, évolution du comportement par tour, performance relative entre stratégies.
- Enrichissez vos données par des colonnes dérivées : mémoire des tours précédents, fréquence de trahison, tendance à pardonner, etc.

1.4.4 Agrégation et exposition

Votre couche Gold expose des tables prêtes à l'analyse, construites à partir de la Silver. Elle ne doit pas contenir de lignes brutes mais uniquement des agrégats, des scores et des vues métier.

Quelques exemples de tables Gold attendues :

- `tournament_summary` : classement final des stratégies avec score total, taux de coopération moyen, ratio victoires/défaites
- `matchup_matrix` : table de confrontation croisée (stratégie A vs B → score moyen des deux côtés)
- `behavioral_drift` : évolution du taux de coopération par tranche de tours (tours 1-100, 101-200...) pour détecter les bascules
- `forgiveness_index` : fréquence de retour à la coopération après une trahison, par stratégie

1.4.5 Choix d'architecture

A l'aide des différentes étapes de projet réalisées en cours, construisez la meilleure architecture de données pour ce projet.

1.4.6 Aller plus loin

La simulation telle que définie produit un jeu de données unique. Mais que se passe-t-il si l'on souhaite rejouer le tournoi des centaines de fois en faisant varier les paramètres - nombre de tours, matrice de gains, composition des stratégies, profils IA ?

Chaque run devient alors un snapshot indépendant. Une nouvelle problématique émerge : comment conserver, comparer et interroger l'ensemble de ces runs sans écraser les résultats précédents ?

Réfléchissez à l'évolution de votre architecture vers un data lake versionné :

- Comment identifier et isoler chaque run (identifiant, horodatage, paramètres) ?
- Comment stocker plusieurs snapshots sans duplication inutile ?
- Comment interroger un run spécifique ou comparer deux runs entre eux ?

1.4.7 Objectif final

Vos données doivent permettre de réaliser une analyse exploratoire par un data analyste :

- Comparaison des stratégies ou profils d'IA ;
- Visualisations (évolution temporelle, heatmaps, corrélations) ;
- Mise en évidence d'un comportement émergent ou d'un équilibre de Nash.

1.5 Livrables

1. **Architecture de données complète** : Dépôt Github
2. **Documentation du projet** : Le projet doit être entièrement répliquable
3. **Données du projet** : Vous ne **DEVEZ PAS** fournir les données du projet
 - Le projet doit être répliquable en suivant uniquement le `README.md`

Rendu sur ce lien : <https://forms.gle/T68K6bhaK3CeP54w8>

1.6 Conclusion

Ce projet vous place dans la position du chercheur : vous créez vos propres données, vous les traitez et les interprétez pour comprendre un phénomène complexe.

Comme Axelrod en 1981, vous devrez décider : serez-vous du côté de la coopération, de la trahison, ou de l'expérimentation ?